



— Exercício: Unidade I —

Alocação dinâmica de memória

Nome: _____ Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade1.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Alocação dinâmica de memória com `malloc` e liberação com `free`;
- Representação de uma **matriz** usando um ponteiro para ponteiro (`int **`);
- Representação de um **vetor dinâmico de structs** (`Bola *`);
- Passagem de **ponteiros para funções**, permitindo modificar o dado original sem copiá-lo;
- Uso de **aritmética de ponteiros** (`bolas + i` equivalente a `&bolas[i]`).

O programa exibe uma janela com uma grade de fundo (matriz) e várias bolinhas coloridas que se movem e quicam nas bordas da tela (vetor de *structs*).

1 Estrutura de dados: a bola

Cada bolinha é representada por uma *struct*, que agrupa posição, velocidade, raio e cor em um único tipo de dado:

```
1 typedef struct {  
2     Vector2 pos;  
3     Vector2 vel;  
4     float   raio;  
5     Color   cor;  
6 } Bola;
```

2 A matriz dinâmica (`int **`)

A grade de fundo é armazenada como uma matriz de inteiros alocada **dinamicamente** em duas etapas: primeiro aloca-se um vetor de ponteiros (um ponteiro por linha) e, em seguida, aloca-se cada linha individualmente como um vetor de `int`.

```
1 int **criarMatriz(int linhas, int colunas) {  
2     int **matriz = (int **)malloc(linhas * sizeof(int *));  
3     for (int i = 0; i < linhas; i++) {  
4         matriz[i] = (int *)malloc(colunas * sizeof(int));  
5         for (int j = 0; j < colunas; j++) {  
6             matriz[i][j] = GetRandomValue(0, 1);  
7         }  
8     }  
9     return matriz;  
10 }
```

Por que duas alocações? Em C não existe um tipo nativo de matriz dinâmica: `matriz` é, na verdade, um vetor de ponteiros, e cada posição desse vetor aponta para um bloco de memória independente (uma linha). Por isso a liberação também precisa de dois passos: primeiro libera-se cada linha, depois o vetor de ponteiros.

```
1 void liberarMatriz(int **matriz, int linhas) {  
2     for (int i = 0; i < linhas; i++) {  
3         free(matriz[i]);  
4     }  
5     free(matriz);  
6 }
```

3 O vetor dinâmico de bolas (Bola *)

Diferentemente da matriz, o vetor de bolas é um único bloco contíguo de memória, alocado com uma só chamada de `malloc`:

```
1 Bola *criarBolas(int quantidade) {
2     Bola *bolas = (Bola *)malloc(quantidade * sizeof(Bola));
3     for (int i = 0; i < quantidade; i++) {
4         Bola *b = (bolas + i); // equivalente a &bolas[i]
5         b->pos = (Vector2){ GetRandomValue(50, LARGURA_JANELA - 50),
6                             GetRandomValue(50, ALTURA_JANELA - 50) };
7         /* ... */
8     }
9     return bolas;
10 }
```

A expressão `bolas + i` é a base da **aritmética de ponteiros**: como `bolas` aponta para o início do bloco alocado, somar `i` desloca o ponteiro em `i` posições de `sizeof(Bola)` bytes, chegando exatamente ao elemento de índice `i` — o mesmo resultado de `&bolas[i]`.

4 Atualização via ponteiro

A função que move cada bola recebe um **ponteiro para a struct**, e não uma cópia dela. Isso é essencial: sem o ponteiro, qualquer alteração feita dentro da função seria perdida ao final da chamada.

```
1 void atualizarBola(Bola *b) {
2     b->pos.x += b->vel.x;
3     b->pos.y += b->vel.y;
4
5     if (b->pos.x - b->raio < 0 || b->pos.x + b->raio > LARGURA_JANELA)
6         b->vel.x *= -1;
7     if (b->pos.y - b->raio < 0 || b->pos.y + b->raio > ALTURA_JANELA)
8         b->vel.y *= -1;
9 }
```

No laço principal, a função é chamada assim:

```
1 for (int i = 0; i < quantidadeBolas; i++) {
2     atualizarBola(bolas + i);
3 }
```

Ciclo de vida da memória

Etapa	Matriz (int **)	Vetor (Bola *)
Alocação	<code>criarMatriz()</code>	<code>criarBolas()</code>
Uso	<code>desenharMatriz()</code>	<code>atualizarBola()</code> + desenho
Liberção	<code>liberarMatriz()</code>	<code>free(bolas)</code>

Toda memória alocada com `malloc` deve, obrigatoriamente, ser liberada com `free` antes do fim do programa — caso contrário, ocorre um *memory leak* (vazamento de memória).

5 Exercícios

Exercício 1 — Redimensionando o vetor em tempo real

Atualmente a quantidade de bolas é fixa (`quantidadeBolas = 12`) durante toda a execução.

Tarefa: modifique o programa para que, ao pressionar a tecla `ESPAÇO`, uma nova bola seja adicionada ao vetor, e ao pressionar a tecla `BACKSPACE`, a última bola seja removida.

Dicas:

- Utilize a função `realloc` para redimensionar o bloco de memória apontado por `bolas`, em vez de recriar o vetor do zero.
- Lembre-se de atualizar a variável `quantidadeBolas` sempre que o vetor mudar de tamanho.
- Use `IsKeyPressed(KEY_SPACE)` e `IsKeyPressed(KEY_BACKSPACE)` do raylib para detectar as teclas.
- Cuidado ao remover a última bola: não é necessário mover elementos, basta diminuir o tamanho do bloco alocado.

Exercício 2 — Colisão entre bolas usando a matriz como "mapa de calor"

Atualmente a matriz (**grade**) serve apenas como fundo decorativo e não interage com as bolas.

Tarefa: modifique o programa para que, toda vez que uma bola passar sobre uma célula da grade, o valor daquela célula na matriz mude de 0 para 1 (marcando a célula como "visitada"), e a cor de desenho dessa célula passe a ser diferente (por exemplo, mais clara).

Em seguida, adicione um contador (variável **int** normal, sem ser ponteiro) que mostra na tela quantas células já foram visitadas por pelo menos uma bola, atualizado a cada quadro.

Dicas:

- Você vai precisar converter a posição (**x**, **y**) em pixels da bola para índices de linha e coluna da matriz. Dica: `coluna = (int)(b->pos.x / TAM_CELULA)`.
- Cuidado com os limites da matriz: verifique se a linha e a coluna calculadas estão dentro do intervalo `[0, linhas)` e `[0, colunas)` antes de acessar `grade[linha][coluna]`, para não acessar memória fora dos limites alocados.
- Para contar as células visitadas, você pode percorrer a matriz inteira a cada quadro (mais simples) ou manter um contador incremental que só aumenta quando uma célula passa de 0 para 1 pela primeira vez (mais eficiente).